

CMPT 354
Database Systems I

The Relational Data Model

Martin Ester
Simon Fraser University
Fall 2023

What is a Data Model?



- A formal notation (language) for describing data.
- *Structure* of the data
 - Conceptual model
 - Higher level of abstraction than data structures in programming languages such as lists or arrays.
- *Operations* on the data
 - Limited set of high level operations: queries and modifications.
 - Speeds-up database programming.
 - Allows DBS to optimize query execution, e.g. choice of most efficient sorting method.
- *Constraints* on the data
 - Capture more of the real world meaning of the data.

Why Study the Relational Model?



- Most widely used model.
 - Vendors: Oracle, IBM, Microsoft, Sybase, etc.
- “Legacy systems” in older models.
 - E.g., IBM’s IMS
- Not so recent competitor: object-oriented model.
 - ObjectStore, Versant, Ontos
 - A synthesis emerging: *object-relational model*
 - ➡ Informix Universal Server, Oracle, DB2
- More recent competitor: semi-structured model.
 - XML

XML



- eXtensible Markup Language
- Inspired by HTML.
- HTML specifies layout on the screen, XML specifies content (semantics) of data.
- Hierarchical data, i.e. records are not flat but can be nested.
- Semi-structured, i.e. data does not need to have a schema.
- Data is self-describing.

XML: Example dataset



```
<bibliography>
```

```
  <book>  <title> Foundations... </title>
```

```
    <author> Abiteboul </author>
```

```
    <author> Hull </author>
```

```
    <author> Vianu </author>
```

```
    <publisher> Addison Wesley </publisher>
```

```
    <year> 1995 </year>
```

```
  </book>
```

```
  ...
```

```
</bibliography>
```

Relational Database: Definitions



- *Relational database*: a set of *relations*.
- *Relation* made up of two parts:
 - *Schema* : specifies name of relation, plus name and type of each column.
 - E.g. Students(*sid*: string, *name*: string, *login*: string, *age*: integer, *gpa*: real).
 - *Instance* : a *table*, with rows and columns, which satisfies the specification of its schema.
#rows = *cardinality*, #columns = *degree / arity*.

Relational Database: Definitions



- Rows are called *tuples* (or *records*), columns called *attributes* (or *fields*).
- Attributes are referenced not by column number, but by name.
 - Order of attributes does not matter.
- Attribute types are called *domains*. Domains consist of atomic values such as integers or strings.
 - No structured values such as lists or sets.
- The order of tuples does not matter, a relation is a set of tuples. The order of tuples resulting from a relational query is undefined.

Relational Database: Definitions



- To put it more formally:
- *Domain*: a set of logically connected values, e.g. *string*, *integer*, *real*.
- *Relation* R :
 k domains $D_1, \dots, D_k$

$$R \subseteq D_1 \times D_2 \dots \times D_k$$

- *Cardinality* = $|R|$
- *Degree / Arity* = k
- A set does not contain duplicates!

Example



Relation Students

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

- Cardinality = 3, degree = 5, all rows distinct.
- Do all columns in a relation instance have to be distinct?

The SQL Language



- Proposed by IBM (system R) in the 1970s.
- Later developed into a standard since relational data model used by many vendors.
- Structured Query Language (SQL):
 - retrieval,
 - insertion, updating, and deletion of data,
 - management and administrative functions.
- All commercial DBSs support SQL, but with proprietary extensions to the standard language.

The SQL Language



- Major versions of the standard:
 - SQL-1986
first version
 - SQL-1992
major revision
 - SQL-1999
triggers, recursive queries, object-oriented features
 - SQL-2003
XML-related features, window functions, etc.
 - SQL-2006
importing XML data, publishing in XML format,
integration of XQuery, etc.

The SQL Language



- Major versions of the standard (contd.)
 - SQL-2008
ORDER BY outside of cursors, TRUNCATE statement, FETCH clause
 - SQL-2011
support for temporal data, PERIOD □ FOR
 - SQL-2016
row pattern matching, polymorphic table functions
 - SQL-2019
multidimensional arrays

The SQL Language



- SQL supports the

- creation

- ```
CREATE <relation name> (<attributes>);
```

- modification

- ```
INSERT INTO <relation name> (<attribute names>)  
VALUES (<attribute values>);
```

- and querying of relations / tables.

- ```
SELECT <attribute names>
FROM <relation names>
WHERE <condition>;
```

- Queries are more complex and will be covered in separate chapter.

# Creating Relations



- **CREATE TABLE** specifies the relation name and its attributes.
- The domain of each attribute is specified, and enforced by the DBMS whenever tuples are added or modified.
- Attributes can have zero or one value from their domain.
- **NOT NULL** specifies that this attribute must have *exactly one* value.

# *Creating Relations*



```
CREATE TABLE Students
 (sid CHAR(20),
 name CHAR(20),
 login CHAR(10),
 age INTEGER,
 gpa REAL);
```

```
CREATE TABLE Enrolled
 (sid CHAR(20),
 cid CHAR(20),
 grade CHAR(2)
 NOT NULL);
```

# Creating Relations



## ■ SQL datatypes (domains):

### ● Character strings

fixed length: CHAR(n) exactly n characters

variable length: VARCHAR(n) up to n characters

### ● Bit strings

fixed length: BIT(n) exactly n bits

variable length: BIT VARYING(n) up to n bits

### ● BOOLEAN

TRUE, FALSE, UNKNOWN

### ● Numbers

INT / INTEGER

FLOAT / REAL

### ● Dates and times (special character strings)

DATE

TIME

# Deleting and Altering Relations



**DROP TABLE** Students;

- Deletes the relation Students. The schema information *and* the tuples (table instance) are deleted.

**ALTER TABLE** Students

**ADD** firstYear INTEGER;

- The schema of Students is altered by adding a new attribute; every tuple in the current instance is extended with a *null* value in the new attribute.

# Adding and Deleting Tuples



- **INSERT INTO** can insert *a single* tuple, by providing its attribute values:

```
INSERT INTO Students (sid, name, login, age, gpa)
VALUES (53688, 'Smith', 'smith@ee', 18, 3.2);
```

- **DELETE** deletes *all* tuples satisfying some condition (e.g., name = Smith):

```
DELETE
FROM Students
WHERE name = 'Smith' ;
```

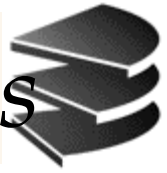
- More powerful variants of these commands are available; more later!

# Integrity Constraints (ICs)



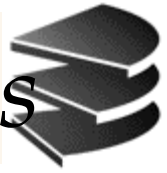
- **IC:** condition that must be true for *any* instance of the database; e.g., *domain constraints*.
  - ICs are specified when schema is defined.
  - ICs are checked when instance is modified.
- A *legal* instance of a relation is one that satisfies all specified ICs.
  - DBMS does not allow illegal instances.
- If the DBMS checks ICs, stored data is more faithful to real-world meaning.
  - Avoids data entry errors, too!

# Primary Keys and Candidate Keys



- A set of attributes is a *key* for a relation if:
  1. No two distinct tuples *can* (!) have same values in all key attributes, and
  2. Condition 1 is not true for any subset of the key.
- A set of attributes is a *superkey* for a relation if condition 1 is fulfilled but condition 2 is not.
- Possibly many *candidate keys* (specified using **UNIQUE**), but exactly one *primary key* (specified using **PRIMARY KEY**).
- DBMS ensures that no two tuples share the same (primary or candidate) key value(s).

# Primary Keys and Candidate Keys



- *Artificial keys* are often introduced, since they are fully under the control of the DBS / the enterprise, e.g., *sid*.
- For each key attribute a value needs to be provided, i.e. a key cannot have the special value *null*.
- Primary key can be used to express references between tables and may also be used to optimize data storage.

# *Primary and Candidate Keys*



```
CREATE TABLE Students
 (sid CHAR(20),
 name CHAR(20),
 login CHAR(10),
 age INTEGER,
 gpa REAL,
);
```

What are keys?

What are superkeys?

Which key to choose as primary key?

# *Primary and Candidate Keys*



```
CREATE TABLE Students
 (sid CHAR(20),
 name CHAR(20),
 login CHAR(10),
 age INTEGER,
 gpa REAL,
 PRIMARY KEY (sid));
```

# Primary and Candidate Keys



“Students can take a course only once. For a given student and course, there is a single grade.”

```
CREATE TABLE Enrolled
(sid CHAR(20)
 cid CHAR(20),
 grade CHAR(2),
);
```

“Students can take only one course, and receive a single grade for that course; further, no two students in a course receive the same grade.”

```
CREATE TABLE Enrolled
(sid CHAR(20)
 cid CHAR(20),
 grade CHAR(2),
);
```

Used carelessly, an IC can prevent the storage of database instances that arise in practice!

# Foreign Keys



- *Foreign key* : Set of attributes in one relation that is used to 'reference' a tuple in another relation. Must correspond to *primary key* of the referenced relation.

→ 'logical pointer'

- E.g. *sid* in Enrolled is a foreign key referring to *Students*:

Enrolled(*sid*: string, *cid*: string, *grade*: string)

# Referential Integrity



- If all foreign key constraints are enforced, *referential integrity* is achieved, i.e., no dangling references.
- Can you name a “data model” without referential integrity?

# Foreign Keys in SQL



- Only students listed in the Students relation are allowed to enroll for courses.

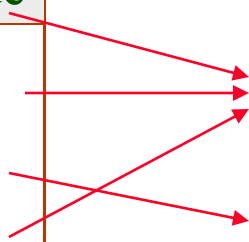
```
CREATE TABLE Enrolled
(sid CHAR(20), cid CHAR(20), grade CHAR(2),
PRIMARY KEY (sid,cid),
FOREIGN KEY (sid) REFERENCES Students);
```

Enrolled

| sid   | cid         | grade |
|-------|-------------|-------|
| 53666 | Carnatic101 | C     |
| 53666 | Reggae203   | B     |
| 53650 | Topology112 | A     |
| 53666 | History105  | B     |

Students

| sid   | name  | login      | age | gpa |
|-------|-------|------------|-----|-----|
| 53666 | Jones | jones@cs   | 18  | 3.4 |
| 53688 | Smith | smith@eecs | 18  | 3.2 |
| 53650 | Smith | smith@math | 19  | 3.8 |



# Enforcing Referential Integrity



- Consider Students and Enrolled; *sid* in Enrolled is a foreign key that references Students.
  - What should be done if an Enrolled tuple with a non-existent *sid* is inserted? *Reject it!*
  - What should be done if a Students tuple is deleted?
    - Also delete all Enrolled tuples that refer to it.
    - Disallow deletion of a Students tuple that is referred to.
    - Set *sid* in Enrolled tuples that refer to it to a *default sid*.
    - In SQL, also: Set *sid* in Enrolled tuples that refer to it to a special value *null*, denoting '*unknown*' or '*inapplicable*'.
- Similar options for updates of primary key of Students.

# Referential Integrity in SQL



- SQL supports all 4 options on deletes and updates.

- Default is **NO ACTION** (delete/update is rejected).
- **CASCADE** (also delete all tuples that refer to deleted tuple).
- **SET NULL / SET DEFAULT** (sets foreign key value of referencing tuple).

```
CREATE TABLE Enrolled
(sid CHAR(20),
cid CHAR(20),
grade CHAR(2),
PRIMARY KEY (sid,cid),
FOREIGN KEY (sid)
REFERENCES Students
ON DELETE CASCADE
ON UPDATE SET DEFAULT);
```

# Where do ICs Come From?



- ICs are based upon the semantics of the real-world enterprise that is being described in the database relations.
- We can check a database instance to see if an IC is violated, but we can *never* infer that an IC is true by looking at an instance.
  - An IC is a statement about *all possible* instances!
  - E.g., we know *name* is not a key, but the assertion that *sid* is a key is given to us.
- Key and foreign key ICs are the most common; more general ICs discussed later.

# Summary



- The relational model is a tabular representation of data. A relation is a subset of the cartesian product of some domains.
- Simple and intuitive, currently the most widely used data model.
- SQL is the standard language for creating, updating and querying relational databases.
- Integrity constraints can be specified by the DBA, based on application semantics. DBMS checks for violations.
- Two most important kinds of ICs: primary and foreign key constraints.
- In addition, we always have domain constraints.